

calorie_requirement_prediction

July 11, 2026

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb
from sklearn.model_selection import train_test_split
sb.set()
```

```
[2]: df = pd.read_csv('healthy_diet_calorie_intake.csv')
```

```
[3]: df.describe(include = 'all')
```

	Person_ID	Age	Gender	Height_cm	Weight_kg	BMI	\
count	6000	6000.000000	6000	6000.000000	6000.000000	6000.000000	
unique	6000	NaN	3	NaN	NaN	NaN	
top	P0001	NaN	Male	NaN	NaN	NaN	
freq	1	NaN	2892	NaN	NaN	NaN	
mean	NaN	48.817167	NaN	168.592533	74.070933	26.094300	
std	NaN	18.141837	NaN	9.160213	12.417877	4.162643	
min	NaN	18.000000	NaN	145.000000	40.000000	12.900000	
25%	NaN	33.000000	NaN	161.700000	65.500000	23.300000	
50%	NaN	48.000000	NaN	168.200000	74.100000	26.000000	
75%	NaN	65.000000	NaN	175.200000	82.700000	28.900000	
max	NaN	80.000000	NaN	198.600000	116.600000	44.000000	

	Activity_Level	Daily_Calorie_Requirement	Daily_Calorie_Consumed	\
count	6000	6000.000000	6000.000000	
unique	5	NaN	NaN	
top	Lightly Active	NaN	NaN	
freq	1627	NaN	NaN	
mean	NaN	2286.223833	2490.520333	
std	NaN	469.541134	602.279679	
min	NaN	1223.000000	1000.000000	
25%	NaN	1941.000000	2057.000000	
50%	NaN	2232.500000	2433.000000	
75%	NaN	2581.250000	2859.000000	
max	NaN	4079.000000	5158.000000	

Protein_Intake_g	Carbohydrate_Intake_g	Fat_Intake_g	\
------------------	-----------------------	--------------	---

count	6000.000000	6000.000000	6000.000000
unique	NaN	NaN	NaN
top	NaN	NaN	NaN
freq	NaN	NaN	NaN
mean	123.773150	288.035317	93.698417
std	61.999158	123.914738	43.953993
min	17.400000	-23.600000	23.200000
25%	78.800000	229.100000	65.900000
50%	108.900000	297.150000	82.300000
75%	155.100000	366.125000	105.100000
max	457.500000	729.700000	341.700000

	Water_Intake_Liters	Diet_Type	Health_Status
count	6000.000000	6000	6000
unique	NaN	6	4
top	NaN	Balanced	Overweight
freq	NaN	1536	2500
mean	2.734917	NaN	NaN
std	0.681486	NaN	NaN
min	1.500000	NaN	NaN
25%	2.200000	NaN	NaN
50%	2.600000	NaN	NaN
75%	3.100000	NaN	NaN
max	5.000000	NaN	NaN

```
[4]: df.head()
```

```
[4]:  Person_ID  Age  Gender  Height_cm  Weight_kg  BMI  Activity_Level \
0      P0001   50   Male    176.4      74.8    24.0      Very Active
1      P0002   18  Female    167.6      75.5    26.9      Sedentary
2      P0003   68  Female    161.9      87.2    33.3      Lightly Active
3      P0004   22  Female    169.3      66.9    23.3  Moderately Active
4      P0005   30   Male    179.1      75.3    23.5      Sedentary
```

	Daily_Calorie_Requirement	Daily_Calorie_Consumed	Protein_Intake_g
0	2852	2625	183.0
1	1904	2044	90.1
2	2009	2540	222.7
3	2318	2096	69.5
4	2144	1937	32.9

	Carbohydrate_Intake_g	Fat_Intake_g	Water_Intake_Liters	Diet_Type
0	16.9	202.8	3.3	Keto
1	306.5	50.8	1.9	Vegan
2	281.3	58.2	2.4	High Protein
3	299.8	68.7	2.9	Balanced
4	285.6	73.7	2.2	Balanced

```

Health_Status
0    Healthy
1  Overweight
2    Obese
3    Healthy
4    Healthy

```

```
[5]: df.shape
```

```
[5]: (6000, 15)
```

```
[6]: df.isnull().sum()
```

```

[6]: Person_ID          0
Age                    0
Gender                0
Height_cm             0
Weight_kg             0
BMI                   0
Activity_Level        0
Daily_Calorie_Requirement  0
Daily_Calorie_Consumed  0
Protein_Intake_g      0
Carbohydrate_Intake_g  0
Fat_Intake_g          0
Water_Intake_Liters   0
Diet_Type             0
Health_Status         0
dtype: int64

```

```
[7]: new_df = df.drop('Person_ID', axis = 1)
```

```
[8]: new_df.head()
```

```

[8]:   Age  Gender  Height_cm  Weight_kg  BMI  Activity_Level \
0   50   Male    176.4    74.8  24.0      Very Active
1   18  Female    167.6    75.5  26.9      Sedentary
2   68  Female    161.9    87.2  33.3  Lightly Active
3   22  Female    169.3    66.9  23.3  Moderately Active
4   30   Male    179.1    75.3  23.5      Sedentary

      Daily_Calorie_Requirement  Daily_Calorie_Consumed  Protein_Intake_g \
0                2852                2625                183.0
1                1904                2044                90.1
2                2009                2540                222.7
3                2318                2096                69.5

```

	4	2144	1937	32.9	
	Carbohydrate_Intake_g	Fat_Intake_g	Water_Intake_Liters	Diet_Type	\
0	16.9	202.8	3.3	Keto	
1	306.5	50.8	1.9	Vegan	
2	281.3	58.2	2.4	High Protein	
3	299.8	68.7	2.9	Balanced	
4	285.6	73.7	2.2	Balanced	

	Health_Status
0	Healthy
1	Overweight
2	Obese
3	Healthy
4	Healthy

```
[9]: new_df.columns
```

```
[9]: Index(['Age', 'Gender', 'Height_cm', 'Weight_kg', 'BMI', 'Activity_Level',
         'Daily_Calorie_Requirement', 'Daily_Calorie_Consumed',
         'Protein_Intake_g', 'Carbohydrate_Intake_g', 'Fat_Intake_g',
         'Water_Intake_Liters', 'Diet_Type', 'Health_Status'],
        dtype='object')
```

```
[10]: cols = ['Gender',
             'Activity_Level', 'Health_Status']
```

```
[11]: cols
```

```
[11]: ['Gender', 'Activity_Level', 'Health_Status']
```

```
[12]: for col in cols:
        print(new_df[col].describe(include = 'unique'))
```

```
count      6000
unique       3
top         Male
freq       2892
Name: Gender, dtype: object
count      6000
unique       5
top      Lightly Active
freq       1627
Name: Activity_Level, dtype: object
count      6000
unique       4
top      Overweight
freq       2500
```

Name: Health_Status, dtype: object

```
[13]: for col in cols:
        print(f"\n{'='*40}")
        print(f"{col}")
        print(f"{'='*40}")
        print(new_df[col].unique())
```

```
=====
Gender
=====
['Male' 'Female' 'Other']

=====
Activity_Level
=====
['Very Active' 'Sedentary' 'Lightly Active' 'Moderately Active' 'Athlete']

=====
Health_Status
=====
['Healthy' 'Overweight' 'Obese' 'Underweight']
```

```
[14]: for col in cols:
        if col == 'Gender':
            new_df[col] = new_df[col].map({'Male' : 0, 'Female' : 1, 'Other' : 2})
        elif col == 'Activity_Level':
            new_df[col] = new_df[col].map({'Sedentary':0, 'Lightly Active' : 1,
↪ 'Moderately Active': 2, 'Very Active': 3, 'Athlete': 4})
        elif col == 'Health_Status':
            new_df[col] = new_df[col].map({'Underweight': 0, 'Healthy': 1,
↪ 'Overweight': 2, 'Obese' : 3})
```

```
[15]: new_df[cols]
```

```
[15]:
```

	Gender	Activity_Level	Health_Status
0	0	3	1
1	1	0	2
2	1	1	3
3	1	2	1
4	0	0	1
...	...	...	...
5995	1	1	3
5996	1	0	2
5997	1	4	3
5998	1	1	2

5999 0 4 2

[6000 rows x 3 columns]

```
[16]: new_df.head()
```

```
[16]:
```

	Age	Gender	Height_cm	Weight_kg	BMI	Activity_Level	\
0	50	0	176.4	74.8	24.0	3	
1	18	1	167.6	75.5	26.9	0	
2	68	1	161.9	87.2	33.3	1	
3	22	1	169.3	66.9	23.3	2	
4	30	0	179.1	75.3	23.5	0	

	Daily_Calorie_Requirement	Daily_Calorie_Consumed	Protein_Intake_g	\
0	2852	2625	183.0	
1	1904	2044	90.1	
2	2009	2540	222.7	
3	2318	2096	69.5	
4	2144	1937	32.9	

	Carbohydrate_Intake_g	Fat_Intake_g	Water_Intake_Liters	Diet_Type	\
0	16.9	202.8	3.3	Keto	
1	306.5	50.8	1.9	Vegan	
2	281.3	58.2	2.4	High Protein	
3	299.8	68.7	2.9	Balanced	
4	285.6	73.7	2.2	Balanced	

	Health_Status
0	1
1	2
2	3
3	1
4	1

```
[17]: new_df['Diet_Type'].unique()
```

```
[17]: array(['Keto', 'Vegan', 'High Protein', 'Balanced', 'Mediterranean',  
       'Vegetarian'], dtype=object)
```

```
[18]: # diet_col = ['Keto', 'Vegan', 'High Protein', 'Balanced', 'Mediterranean',  
#           'Vegetarian']  
  
# diet_map = {diet: i for i, diet in enumerate(diet_col)}  
  
# new_df['Diet_Type'] = new_df['Diet_Type'].map(diet_map)
```

```
[19]: new_df.head()
```

```
[19]:
```

	Age	Gender	Height_cm	Weight_kg	BMI	Activity_Level	\
0	50	0	176.4	74.8	24.0	3	
1	18	1	167.6	75.5	26.9	0	
2	68	1	161.9	87.2	33.3	1	
3	22	1	169.3	66.9	23.3	2	
4	30	0	179.1	75.3	23.5	0	

	Daily_Calorie_Requirement	Daily_Calorie_Consumed	Protein_Intake_g	\
0	2852	2625	183.0	
1	1904	2044	90.1	
2	2009	2540	222.7	
3	2318	2096	69.5	
4	2144	1937	32.9	

	Carbohydrate_Intake_g	Fat_Intake_g	Water_Intake_Liters	Diet_Type	\
0	16.9	202.8	3.3	Keto	
1	306.5	50.8	1.9	Vegan	
2	281.3	58.2	2.4	High Protein	
3	299.8	68.7	2.9	Balanced	
4	285.6	73.7	2.2	Balanced	

	Health_Status
0	1
1	2
2	3
3	1
4	1

```
[20]: new_df = pd.get_dummies(new_df, columns = ['Diet_Type'], drop_first = True,
↳ dtype = int)
```

```
[21]: new_df.head(10)
```

```
[21]:
```

	Age	Gender	Height_cm	Weight_kg	BMI	Activity_Level	\
0	50	0	176.4	74.8	24.0	3	
1	18	1	167.6	75.5	26.9	0	
2	68	1	161.9	87.2	33.3	1	
3	22	1	169.3	66.9	23.3	2	
4	30	0	179.1	75.3	23.5	0	
5	36	0	175.7	73.2	23.7	4	
6	29	0	172.3	93.6	31.5	1	
7	65	1	164.9	80.1	29.5	2	
8	42	1	159.5	80.7	31.7	0	
9	74	1	156.9	67.5	27.4	1	

	Daily_Calorie_Requirement	Daily_Calorie_Consumed	Protein_Intake_g	\
0	2852	2625	183.0	

1	1904	2044	90.1
2	2009	2540	222.7
3	2318	2096	69.5
4	2144	1937	32.9
5	3245	3434	268.3
6	2756	3147	312.3
7	2197	2609	108.3
8	1807	2180	216.4
9	1701	2008	60.5

	Carbohydrate_Intake_g	Fat_Intake_g	Water_Intake_Liters	Health_Status \
0	16.9	202.8	3.3	1
1	306.5	50.8	1.9	2
2	281.3	58.2	2.4	3
3	299.8	68.7	2.9	1
4	285.6	73.7	2.2	1
5	333.0	114.3	4.5	1
6	267.7	91.9	2.6	3
7	412.6	58.4	2.6	2
8	194.7	59.5	2.5	3
9	271.6	75.5	2.4	2

	Diet_Type_High Protein	Diet_Type_Keto	Diet_Type_Mediterranean \
0	0	1	0
1	0	0	0
2	1	0	0
3	0	0	0
4	0	0	0
5	1	0	0
6	1	0	0
7	0	0	0
8	1	0	0
9	0	0	1

	Diet_Type_Vegan	Diet_Type_Vegetarian
0	0	0
1	1	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0
7	1	0
8	0	0
9	0	0

```
[22]: from sklearn.model_selection import train_test_split
```

```
[23]: inputs = new_df.drop(columns=[
    'Daily_Calorie_Requirement',
    'BMI',
    'Daily_Calorie_Consumed',
    'Protein_Intake_g',
    'Carbohydrate_Intake_g',
    'Fat_Intake_g'
])
```

```
[24]: target = new_df['Daily_Calorie_Requirement']
```

```
[25]: x_train, x_test, y_train, y_test = train_test_split(inputs, target, test_size = 0.2, random_state = 42)
```

```
[26]: x_train
```

```
[26]:
```

	Age	Gender	Height_cm	Weight_kg	Activity_Level	Water_Intake_Liters	\
3897	59	1	163.8	65.0	2	2.6	
5628	75	1	168.7	60.7	1	2.7	
1756	28	1	165.6	66.9	2	2.9	
2346	52	1	156.7	49.4	0	2.4	
2996	68	0	173.0	85.2	1	2.3	
...	...	...	...	...	...	...	
3772	49	0	168.5	83.2	2	2.8	
5191	36	1	154.9	74.3	1	2.4	
5226	51	1	155.1	60.5	1	2.3	
5390	30	0	181.2	66.2	0	2.1	
860	23	1	156.4	55.1	2	2.7	

	Health_Status	Diet_Type_High Protein	Diet_Type_Keto	\
3897	1	0	0	
5628	1	0	0	
1756	1	0	0	
2346	1	0	0	
2996	2	1	0	
...	...	...	...	
3772	2	0	0	
5191	3	0	0	
5226	2	0	0	
5390	1	0	0	
860	1	0	0	

	Diet_Type_Mediterranean	Diet_Type_Vegan	Diet_Type_Vegetarian
3897	0	0	0
5628	0	0	1
1756	0	0	1
2346	0	1	0

2996	0	0	0
...	...	...	...
3772	0	0	0
5191	0	0	1
5226	0	0	1
5390	0	0	1
860	0	0	1

[4800 rows x 12 columns]

```
[27]: y_train
```

```
[27]: 3897    2016
      5628    1659
      1756    2260
      2346    1398
      2996    2302
```

```
      ...
      3772    2687
      5191    2006
      5226    1742
      5390    2009
      860     2080
```

Name: Daily_Calorie_Requirement, Length: 4800, dtype: int64

```
[28]: from sklearn.ensemble import RandomForestRegressor
```

```
[29]: rf_model = RandomForestRegressor(n_estimators = 100, random_state = 42)
```

```
[30]: rf_model.fit(x_train, y_train)
```

```
[30]: RandomForestRegressor(random_state=42)
```

```
[31]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
y_pred = rf_model.predict(x_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"Mean Absolute Error (MAE): {mae:.2f} kcal")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f} kcal")
print(f"R² Score: {r2:.4f}")
```

Mean Absolute Error (MAE): 25.93 kcal

Root Mean Squared Error (RMSE): 39.33 kcal

R² Score: 0.9926

```
[32]: import joblib
```

```
[33]: joblib.dump(rf_model, 'calorie_requirement_prediction_RF.joblib')
```

```
[33]: ['calorie_requirement_prediction_RF.joblib']
```